Biopython

Contents

What is Biopython?

Biopython I [1]

- A set of freely available tools for biological computation
- It is a distributed collaborative effort to develop Python libraries and applications which address the needs of current and future work in bioinformatics
- Code to deal with popular on-line bioinformatics destinations such as:
  - NCBI – Blast, Entrez and PubMed services
  - ExPASy – Swiss-Prot and Prosite entries, as well as Prosite searches
- Interfaces to common bioinformatics programs such as:
  - Standalone Blast from NCBI
  - Clustalw alignment program
  - EMBOSS command line tools

Biopython II [1]

- Allows you to parse / read in bioinformatics files into Python utilizable data:
  - Blast output – both from standalone and WWW Blast
  - Clustalw
  - FASTA
  - GenBank
  - PubMed and Medline
  - ExPASy files, like Enzyme and Prosite
  - SCOP, including ‘dom’ and ‘lin’ files
  - UniGene
  - SwissProt

Biopython III [1]

- It defines a standard sequence class that deals with sequences, ids on sequences, and sequence features
- Contains tools for performing common operations on sequences, such as translation, transcription and weight calculations
- Contains code for dealing with alignments, including a standard way to create and deal with substitution matrices
- Offers code making it easy to split up parallelizable tasks into separate processes
- It’s functionally similar to other Bio* projects, such as BioPerl for example

Biopython IV [2]

- Biopython contains a multitude of key features, which are shown down below
- During this course we will discuss at least some of them

- Seq class
- SeqRecord class
- SeqIO parser
- Entrez
- Phylogenetic analysis
- Genome Diagrams
- PDB module
- statistical analysis of Population genetics
- Wrappers for command line tools

Insert: XML

XML I – Basics

- XML stands for extensible mashup language and allows us to create xml files
- These files are similar to html files in their structure
- They consist of three elements
  1. The tags, the structure elements of our files
  2. the attributes, additional information added to our tags
  3. the values, the data we want to save in our files

XML II – Example File


```
<protein_list>
    <protein>
        <name>RNA-Polymerase</name>
        <weight unit="Da">50473</weight>
    </protein>
    <protein>
        <name>Helicase</name>
        <weight unit="Da">105478</weight>
    </protein>
</protein_list>
```

opening tag

value

attribute

closing tag

XML III – Reading in

- When working with xml files there are different ways to read them in
- However in most cases the functions generate a nested dictionary out of these files where each tag becomes a key
- For the example we saw before our dictionary could look as follows:

```
dict = {"protein_list":
            [{"name":"RNA-Polymerase","weight":
                  {"attribute":"DA","value":50473}},
             {"name":"Helicase","weight":
                  {"attribute":"DA","value":105478}}
            ]}
```

XML IV – Maneuvering through the dictionary

- Once we read in the xml-file we can maneuver through the dictionary
- A good way of doing so is to increase the used keys and/or indices step by step:

```
print(dict["protein_list"])

print(dict["protein_list"][0])

print(dict["protein_list"][0]["weight"])

print(dict["protein_list"][0]["weight"]["value"])
```

- If you're unsure what keys are available to you, you can use the keys() method in between:

```
print(dict["protein_list"][0].keys())
```

Entrez and Biopython Part I

Biopython Entrez I [3]

- We will first focus on using the web services of the Biopython module in combination with the Entrez NCBI database system
- To be able to use these functions we need the following import:
  - `from Bio import Entrez`
- There are several functions associated with this sub module and we will go over them one by one
- For each sub module of biopython we need an import similar to the one shown above

Biopython Entrez II [3]

- When working with the Entrez submodule we have to hand over an email to the NCBI in case there are some return messages
  - `Entrez.email =„“`
- This is mainly for sending information about server downtimes and things like this to the users
- In all the years I worked with this module I never got back any E-Mails
- However if you forget to specify an E-Mail address you won‘t be able to run the different functions

Biopython Entrez III [3]

- When working with the different functions from the Entrez sub module we always have to keep in mind that the servers send us text files, which we need to read in to be able to access the data written in them
- Per default most of these files are in the xml format, a certain informatic format, that is mainly converted to nested dictionaries during the reading in process
- We will talk more in depth about the process of parsing such files by hand in the third module
- For now we will mainly focus on the built-in functions of Bio.Entrez or the SeqIO parser to access the data we get back from the NCBI

Einfo I [3]

- The first function Einfo is mainly used to help you later on with creating good search queries with other functions
- By using this function we are able to get more in depth information about a database of our choice
- When using the functions from this module we always need a so called with statement that allows us to send a request to the NCBI server
- Since we are working with functions that are part of the Entrez submoldue of biopython we need to use the point notation for these functions

Einfo II [3]

- Now we can create our with statement to send a request to the NCBI as follows

```
with Entrez.einfo() as query:
```

- First we have the keyword with
- Than comes the function we want to use, here it‘s einfo() without any arguments
- Afterwards we have to use the keyword as and specify a name, that is than given to the answer from the database, that means this variable later on contains the file we get back from the NCBI
- Of course this name is freely choosable, but conventions are to use either query or handle
- Don‘t forget to close the line with a colon :

Einfo III [3]

- As we learned, once a line ends with a colon, we always need another line of code directly afterwards
- When working with biopython, this line of code mostly contains the command for reading the file we got back from the NCBI
- We can do this with the help of the Entrez function read():

```
with Entrez.einfo() as query:
    parse_dict = Entrez.read(query)
```

- As you can see this function takes the name of our file, here query as argument and assigns the result of reading in the contents to a new variable here named parse_dict

Einfo IV [3]

- If you want to know how the different databases are named you can use einfo() without arguments and you can get access to a list of all databases with the help of the key “DbList” in the dictionary Entrez.read created for you:

```
with Entrez.einfo() as query:
        parse_dict = Entrez.read(query)
print(parse_dict[“DbList”])
```

- On the other hand when specifying a database, one of the interesting things we can look up in the resulting dictionary is the so called field list
- This list contains all potential search field, e.g. author or organism, we could use to create more complex searches later on

Einfo V [3]

- The function Entrez.read() always creates a dictionary, which contains all information contained in the file we got back
- Please keep mind, that you can read in these files only once for each with statement
- That means if you want to read in the same answer from the NCBI with different functions, you would need one with statement for each function you want to try out

Einfo VI [3]

- As we saw, we can use einfo() without any arguments, in this case we get back a list of databases available at the NCBI server
- In most cases however we are more interested in getting information about a certain server instead
- In these cases we can add the optional argument db and specify for which database we want to look up more information:

```
with Entrez.einfo(db="pubmed") as query:
    PubMedDict = Entrez.read(query)
```

- In the example above I request information about the database pubmed, as you can see the name differs from the one we saw on the web

Einfo VII [3]

- We can access this list by using the keys “DbInfo” and “FieldList”:

```
with Entrez.einfo(db="pubmed") as query:
        PubMedDict = Entrez.read(query)


for field in PubMedDict["DbInfo"]["FieldList"]:
        print(field["Name"],field["FullName"],field["Description"])
```

- In the example above I‘m asking for information about the PubMed and afterwards I loop through the dictionary that is the value of the key “FieldList”

Entrez.read()

- The function Entrez.read() we use to read in the answers from the NCBI ONLY works with XML files
- Later on we can get back other files, e.g. fasta files, gene tables, abstracts etc., in all of these cases we can NOT use ez.read()

Exercise 06: using Entrez.einfo

- Use Entrez.einfo to make queries about the following databases of the NCBI Entrez system:
  - Gene
  - Nucleotide
  - Genome
  - Protein
  - Structure
- Loop through Dict["DbInfo"]["FieldList"] to see what fields are available to you for searching in these databases

ESearch I [3]

- We saw how to use einfo() to get information about specifc databases
- Now we can use this knowledge to formulate precise searches in the NCBI databases
- We can do so by using the function esearch() of the Entrez submodule
- This function has at least two arguments we need to specify:
  - db ➔ the database we want to search in
  - term ➔ The things we want to search for in case insensitive matter
- When using this function we get back a list of identifiers for the different entries in the database that fulfill our search criteria

Esearch II [3]

- Let‘s say we want to search in PubMed for articles that contain the word biopython in their title
- We could do so by using esearch() as follows:

```
with Entrez.esearch(db="pubmed",term="biopython[title]") as query:
    record_pub = Entrez.read(query)
```

- As you can see I specify the database I want to search in by handing over the string “pubmed” to the argument db
- I also specify my search term by handing over the string “biopython[title]” to the argument term, this means I’m searching in PubMed for entries that contain the word Biopython in the title

Esearch III [3]

- The keyword title we saw in square brackets in the search term of the last example is called a field in biopython and refers to the advanced search options available for the databases
- The names of these fields are case insensitive
- Depending on the database these fields differ, as you could see when we were talking about einfo
- When working with esearch we have to keep some things in mind:
  1. We only get back the first twenty results of our search
  2. We get back the database internal identifiers, which are just ongoing numbers, and not identifiers like accession numbers that could be used in multiple databases at once

Esearch IV [3]

- If we want to work with more than just the first twenty results of a search we can add the argument retmax to our esearch function, to specify how many results we want to get back instead:
  - `Entrez.esearch(db="pubmed",term="Polymerase[title]",retmax=50)`
- In the example above I would get back 50 identifiers instead of just 20
- The maximum is 10000 identifiers, that means if your search produces more than that, you either need to adjust your search or work around the maximum (next slide)

Esearch V [3]

- If your search produces for example 25000 results you would need to three separate searches as shown below
- I added the optional argument retstart, that tells biopython which found identifier I would like to handed back to me as the first one (index starts at 0)
- So in the example below, the first search returns the first 10K, the second search starts with identifier 10000 and hands me back the next 10k and so on

```
Entrez.esearch(db="pubmed",term="Polymerase[title]",retmax=10000)
Entrez.esearch(db="pubmed",term="Polymerase[title]",retmax=10000,retstart=10000)
Entrez.esearch(db="pubmed",term="Polymerase[title]",retmax=10000,retstart=20000)
```

Esearch VI [3]

- If you want to use the accession numbers, which are identifiers used by multiple databases to refer to the same data, you can do so by using the optional argument idtype
- The advantage of these accession numbers is that you can search in Nucleotide for example and you would be able to find the corresponding protein in the database protein under the same accession number, whereas the normal identifiers only work on the database they stem from:

```
Entrez.esearch(db="nucleotide",term="Polymerase[title]",idtype='acc')
```

- To switch the identifier type simply hand over the value 'acc' to the optional argument idtype as shown above

Esearch VII [3]

- When creating our search term we are NOT limited to using only one field
- We can combine multiple fields and search terms with the help of logical operators and / or / not
- If we want to search for the matK gene in Slipper orchids we could do so as shown below:

```
Entrez.esearch(db="nucleotide", term="Cypripedioideae[Orgn] AND matK[Gene]")
```

- Keep in mind when working with organisms that we need to use the latin name for the organism
- As you can see I used the abbreviation for the search fields (Orgn) instead of the whole name (Organism) which works just fine

Esearch VIII [3]

- You can use multiple words for a singular search field as shown below:

```
Entrez.esearch(db="nucleotide", term='homo sapiens[Organism]’)
```

- However to be on the save side it’s better to include such terms in double quotation marks “ “

```
Entrez.esearch(db="nucleotide", term=‘”homo sapiens”[Organism]')
```

Esearch IX [3]

- The results you‘re getting back are sorted according to the default sort order of the database you searched in
- However these databases also allow you to sort according to different criteria
- To make use of these options you have to include the optional argument sort and hand over a string for the sort criterium:

```
Entrez.esearch(db="pubmed",term="monkeypox[title]",sort="journal")
```

- To see which options are available for the database you‘re working with, you need to go to the results pages of the corresponding database

Esearch X [3]

- As mentioned the values you can use for sort depend on the database you work with
- For PubMed you have:
  - pub_date – publication date (descending)
  - Author – first author (ascending)
  - JournalName – journal name (ascending)
  - relevance – default sort order
- Other databases have:
  - pdat – publication date
  - slen – sequence length

Exercise 07: Searching NCBI databases

- Write a script that is able to run at least five of the searches summarized in the table below:

Database	Search term(s)
PubMed	title: Hepatitis C NS5B & title: inhibitor
Gene	all: Wnt Pathway
Genome	title: Monkeypox
Pubmed	title: Monkeypox & publication date: 2022
Protein	all: Borna Virus & sequence length: 250 – 500
BioSample	title: Varicella-zoster virus
Structure	title: Pyruvate Kinase & EXPM: electron microscopy

Esummary I [3]

- Depending on your task and search it might be necessary to filter the results you got back from your search before starting to download the complete entries from the database
- In these cases the function esummary() can come in handy, since she allows us to get back only a short summary of the corresponding entry, including essential information like author(s), title, publication date and so on for one entry in our database
- Esummary takes two arguments we always need to specify:
  1. db ➔ the database were our id stems from
  2. id ➔ the id of the entry we want a summary for

Esummary II [3]

- The general approach for using esummary() is by using a with statement using the esummary() function:

```
with Entrez.esummary(db="pubmed", id="18606172") as handle:
    record = Entrez.read(handle)
```

- esummary always hands us back xml-files, therefore we need the Entrez.read() function to convert it to a python-useable datastructure

Esummary III [3]

- Please be aware that the variable record can be a list that has exactly one element and this element is a dictionary, which contains the necessary data of our summary
- So we need an index and keys to access the title for example:

```
record[0]["Title"]
```

Exercise 08: Creating Summaries

- Use some of the IDs you can see in the table below to create summaries of them from the corresponding database

ID(s)	Database
25562845	PubMed
21926, 406991 & 595	Gene
82219	Genome
37426289 & 36814644	PubMed
215981657 & 215981655	Protein
23498700 & 4623491	Biosample
231743	Structure

Working with sequences

SeqIO I [3]

- When working with fasta or genbank files we can use a different submodule of biopython called Sequence Input/Output, in short SeqIO, that can help us with the reading of of these files
- To use this submodule we need the following import:

```
from Bio import SeqIO
```

- For now we will focus only on the read function of the submodule SeqIO, that is able to read in a wide variety of bioinformatic files
- You can find more information about this submodule and the compatible file types here:
  - https://biopython.org/wiki/SeqIO

SeqRecord class I

- The SeqIO.read() function generates so called SeqRecord objects
- These objects and especially the sequence objects in them have some nice bioinformatic functionality, which I want to explain briefly here
- To be able to work with such an object, we need to read in a file first of course, and for this example we will work with a simple fasta file containing a DNA sequence:

```
record1 = SeqIO.read('AH014196.fasta','fasta')
```

- Now we created a SeqRecord object that contains an object of the Seq class which we will focus on here

SeqRecord class II

- To access the Seq object in our SeqRecord object we need to use the already known attribute .seq:

```
record1.seq
```

- The first function of these Seq objects I want to discuss here is the function transcribe(), that allows us to generate a RNA sequence out of the DNA sequence we got here:

```
rna_record1 = record1.seq.transcribe()
```

- rna_record1 is now a Seq object containing the RNA sequence corresponding to the DNA sequence saved in record1.seq

SeqRecord class III

- The second function of these Seq objects I want to discuss here is the function back_transcribe(), that allows us to generate a DNA sequence out of the RNA sequence we got here:

```
dna_record1 = rna_record1.back_transcribe()
```

- dna_record1 is now a Seq object containing the DNA sequence corresponding to the RNA sequence saved in rna_record1

SeqRecord class IV

- Additional functions that could be useful are complement() and reverse_complement() as well as complement_rna() and reverse_complement_rna()
- These produce the (reverse) complement sequence for a given DNA or RNA sequence:

```
dna_complement1 = dna_record1.complement()

dna_reverse_complement1 = dna_record1.reverse_complement()

rna_complement1 = rna_record1.complement_rna()

rna_reverse_complement1 = rna_record1.reverse_complement_rna()
```

SeqRecord class V

- When we have a Seq object with a nucleotide sequence we can use another function to get the corresponding protein sequence
- This function is called translate and we can use it as follows:

```
protein_record1 = rna_record1.translate()
```

- Here we translate our RNA sequence into a protein sequence
- The whole process starts at the first codon and goes on until it reaches the last one, without taking into account any non-coding regions and so on
- So this is just a very rudimentary way of translating nucleotide sequences into protein sequences
- If there are any Stop codons in your sequence, you will get a * instead of a letter

SeqRecord class VI

- If you want the translation of a RNA sequence to stop at the first occurence of a stop codon you can add the argument to_stop=True to the translate function:

```
protein_record2 = rna_record1.translate(to_stop=True)
```

Entrez and Biopython Part II

Efetch I [3]

- After searching for entries that fulfill certain criteria with esearch and filtering them using esummary we can now start to download the data from the NCBI databasese using the function efetch
- This function needs us to specify four different arguments to be able to download the data into our python script:

1. db ➔ The database we want to download data from
2. id ➔ The identifier of the entry we want to download
3. retmode ➔ The datatype that is returned to us
4. rettype ➔ The type of data that is returned to us

Efetch II [3]

- When working with efetch() different databases have different available combinations of the arguments retmode and rettype and you can find a tabular overview of allowed combinations here:
  - https://www.ncbi.nlm.nih.gov/books/NBK25499/table/chapter4.T._valid_values_of__retmode_and/?report=objectonly
- All databases however allow us to get back files in the xml format by using only the argument retmode="xml" and leaving out the argument retype

Efetch III [3]

- Let‘s say I want to download an entry from nucleotide in the default xml format, with the help of efetch
- In this case my with statement needs to look like this:

```
with Entrez.efetch(db="nucleotide", id="EU490707", retmode="xml") as handle:
```

- This statement tells biopython I want to download something from nucleotide (db), the entry I want to download (id) and the file format, here xml, of my download (retmode)

Efetch IV [3]

- Inside my with statement I can now use the already known Entrez.read() function to read in the corresponding xml file and create a big datastructure out of it:

```
with Entrez.efetch(db="nucleotide", id="EU490707", retmode="xml") as handle:
    record1 = Entrez.read(handle)
```

- The exact structure that is created by Entrez.read() depends on the databases and for most of them we create a list with one element and this element is a big nested dictionary

Efetch V [3]

- When looking at the table of potential retmode and rettype combinations, we can see that there are a lot of different datatypes
- When working with sequence data the file types fasta and genbank can be especially useful, since they are two main file types used in bioinformatics when working with sequences
- We will talk more in-depth about these file formats in the third module, for now I want to focus on reading them in using biopython

SeqIO II [3]

- We can use the SeqIO.read() function instead of the Entrez.read() function inside our with statements to read in the corresponding files
- However we need to adjust the combination of retmode and rettype inside our efetch function to do so:

```
with Entrez.efetch(db="nucleotide",id="EU490707",rettype="gb",
retmode="text") as handle:
```

- The example above allows me to download the same entry as bevor but this time I‘m downloading a genbank file indicated by the retmode=“text” and rettype=“gb”

SeqIO III [3]

- Now I can use the SeqIO.read() function inside my with statement
- This function takes two arguments, first the file you want to read in, could also be a file on your hard drive, second the format of your file as string:

```
with Entrez.efetch(db="nucleotide", id="EU490707", rettype="gb",
retmode="text") as handle:
    record2 = SeqIO.read(handle, "genbank")
```

- record2 is now an object of the SeqRecord class which contains a lot of methods and attributes that can be very useful
- The two most useful attributes are probably `record2.id` which contains the identifier and `record2.seq` which contains the sequence of our entry

SeqIO IV [3]

- As I mentioned we can also work with fasta files and when using these kind of files we would need to adjust our program to look as follows:

```
with Entrez.efetch(db="nucleotide", id="EU490707", rettype="fasta",
retmode="text") as handle:
    record3 = SeqIO.read(handle, "fasta")
```

- The general syntax is identical to before, but this time I use “fasta” for the rettype as well as argument for the SeqIO.read() function
- We still have access to the two attributes `record3.id` and `record3.seq` to get the identifier of our entry and the sequence respectively

Exercise 09: Fetching Data

- Use some of the IDs from Exercise 8 to download data from the different databases

Elink I [3]

- When talking about biological databases I mentioned that we need especially interlinked databases for in-depth biological research
- As we saw during our work with the NCBI databases these databases are interlinked, think back to the available crosslinks
- So it would be good to make use of these crosslinks in our python scripts as well
- To do this we can use the function elink() which allows us to search for possible crosslinks in another database for a specific entry

Elink II [3]

- We can use this function for example to find a nucleotide sequence which encodes a certain protein sequence, for which we know the Protein identifier
- To do this we can create a with statement as follows:

```
protid = "1479555242"
with Entrez.elink(db="nucleotide",dbfrom="protein",id=protid) as query:
    record = Entrez.read(query)
```

- As you can see elink() takes three arguments that have a distinct meaning:
  - db ➔ The database in which we want to search for crosslinks
  - dbfrom ➔ The database our identifier stems from
  - id ➔ The identifier of an entry we want to find crosslinks for

Elink III [3]

- There are different types of crosslinks we can get back when using elink and all of them are collected in the datastructure Entrez.read() creates
- To get access to a list containing several dictionaries, one for each crosslink type, we have to the following combination of indices and keys:

```
record[0]['LinkSetDb‘]
```

- If no crosslinks could be found this list is empty
- Depending on the database combination, assuming we found at least one crosslink, this list contains at least one dictionary and we can access a list of dictionaries containing all found identifiers, for the first type, as follows:

```
record[0]['LinkSetDb'][0]['Link']
```

Exercise 10: Linked data

- Use some of the IDs you found during Exercise 7 to search for crosslinks between different databases

Multiple Ids I

- You saw how certain functions of the entrez submodule use the Ids you found using esearch to allow you to deal with the data of the corresponding entry in different ways:
  - esummary
  - efetch
  - elink

Multiple Ids II

- All of these functions also allow you to use a list of Identifiers instead of singular Identifiers (next slide)
- Of course this can make your life easier, since you don‘t need a loop to go through each identifier manually
- However the resulting data structures can be very big and the whole download process can also take a long time
- Therefore I would advise you to use this only with small lists, maybe around 20 identifiers at maximum

Multiple Ids III

- For example you can create multiple summaries at once by using a list of identifiers as value for the keyword argument id:

```
id_list = ["19304878", "18606172", "16403221", "16377612”]


with Entrez.esummary(db="pubmed", id=id_list) as handle:
        record_summary = Entrez.read(handle)
```

- record_summary would now be a list containing four dictionaries, one for each ID in the list

UniProt

UniProt I

- Biopython contains two submodules which you can combine to read in data from the UniProt
- These submodules are called ExPASy and SwissProt and need to be imported as follows:

```
from Bio import ExPASy
from Bio import SwissProt
```

UniProt II

- Once you imported these submodules you can make use of the get_sprot_raw() function of ExPASy to get access to a single UniProt Entry
- This function takes a UniProt identifier as argument in form of a string:

```
handle = ExPASy.get_sprot_raw("O23729")
```

- handle is a so-called TextWrapper object that needs to be read in separately afterwards

UniProt III

- Afterwards you can use the function read() from the submodule SwissProt to read in the sequence you got back via ExPASy
- To do so you need to hand over the TextWrapper object you created earlier as argument to the read() function:

```
record = SwissProt.read(handle)
```

- record is now a SwissProt record object and you can access the sequence by using the attribute sequence of this object:

```
record.sequence
```

RCSB-PDB

RCSB-PDB I

- Similar to the interfaces to the NCBI and the SwissProt biopython also contains an interface to the RCSB-PDB
- This interface is part of the PDB submodule of biopython
- To make use of it we need the function PDBList :

```
from Bio.PDB import PDBList
```

- Furthermore you need either PDBParser (from the sub-submodule PDBParser) or MMCIFParser (depending on the file type) which can be imported as follows:

```
from Bio.PDB.PDBParser import PDBParser
from Bio.PDB.MMCIFParser import MMCIFParser
```

RCSB-PDB II

- The first two steps when accessing files from the RCSB-PDB are to create a PDBList object

```
pdbl = PDBList()
```

- Once this object was created youc an use the retrieve_pdb_file() method of this object to download a file from the RCSB-PDB

```
filename = pdbl.retrieve_pdb_file("1FAT")
```

- This method takes one argument, the PDB-Identifier of the corresponding file, and downloads the corresponding structure file
- Per default you get back a mmcif-file and not a pdb-file

RCSB-PDB III

- If you want to work with a pdb-file instead of a mmcif-file you need to add the optional argument file_format to the retrieve_pdb_file() method as follows:

```
filename = pdbl.retrieve_pdb_file("1FAT",file_format='pdb‘)
```

- The file which is downloaded is than in the pdb-format but has the extension .ent

RCSB-PDB IV

- Once you downloaded the file you can read it in
- Independent of the file-format the first thing you need to do is to create a parser() object by using the PDBParser or MMCIFParser function:

```
parser = PDBParser()

parser = MMCIFParser()
```

- Afterwards you can use the parser object to read in the file you just downloaded

RCSB-PDB V

- The parser objects have a method called get_structure() which allows you to read in the a file of the corresponding format and create a Bio.PDB.Structure object out of it:

```
structure = parser.get_structure("1fat", filename)
```

- The get_structure() method takes two arguments:
  - An identifier you can freely choose to name your Structure object
  - The name of the file you want to read in (here the path PDBList gave us back, after the file was downloaded

Literature

1. https://biopython.org/ (Last edited: 18.11.2022, Last visited: 09.01.2023)
2. https://en.wikipedia.org/wiki/Biopython (Last edited: 24.04.2023, Last visited: 25.04.2023)
3. http://biopython.org/DIST/docs/tutorial/Tutorial.html#sec143 (Last edited: 18.11.2022, Last visited: 09.01.2023)